

STUDENT PERFORMANCE ANALYZER

A Python-Based Student Academic Performance Management System

- Name: Pranav Verma
- Registration/Roll number: 26BCE11478
- Course: B.Tech CSE
- College: VIT Bhopal University



PROJECT OVERVIEW

INTRODUCTION

The Student Performance Analyzer is a Python-based console application for managing and analyzing student academic performance.

- Add student details and marks
- Calculate total and average marks
- Assign grades and determine pass/fail status
- Display records, find the topper, search students, and generate class statistics



MANUAL MARK MANAGEMENT CREATES AVOIDABLE EFFORT

The project replaces repetitive calculations with a simple automated workflow that stores records during execution and produces results consistently.

- Reduce manual calculation errors
- Identify the topper quickly
- Search student records instantly
- Generate overall class statistics

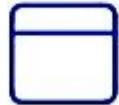
OBJECTIVE

Create a simple, efficient, and user-friendly Python application for analyzing student academic performance.

SIX FUNCTIONS COVER THE COMPLETE WORKFLOW

- | | |
|---|--|
| 1 Add Student: accept name, M1, M2, and M3 marks; calculate total, average, grade, and result. | 2 Display Students: show registered students in a formatted table. |
| 3 Find Topper: identify the student with the highest average. | 4 Search Student: find an individual record by name. |
| 5 Class Statistics: calculate total students, pass/fail counts, class average, and pass percentage. | 6 Exit: terminate the program. |

SIMPLE DESIGN KEEPS THE ANALYZER DEPENDABLE



Usability:

simple menu-driven interface that beginners can understand



Performance:

quick calculations for small student datasets



Reliability:

predefined calculation logic produces consistent results



Maintainability:

separate functions keep each task focused



Portability:

runs wherever Python is installed without external libraries

THE PROGRAM TURNS INPUT INTO ACTIONABLE OUTPUT

Use a clear end-to-end flow diagram from user input to output.

- 1 User opens the main menu and enters student details.

- 2 The Python program runs **calculate_result()**.

- 3 The result is stored in a student dictionary inside the in-memory **students** list.

- 4 The list powers display, topper, search, and statistics operations.

ONE MENU SUPPORTS EVERY USER ACTION

Workflow

Actor: User

Workflow: Start → Display Menu → Select Option → Perform Operation → Display Result → Return to Menu or Exit

ER diagram: Not applicable because the current version uses temporary in-memory storage rather than permanent database/file storage.

Use cases

Use cases: Add Student, Display Students, Find Topper, Search Student, View Class Statistics, Exit

Function/component view

Function/component view: main() connects add_student(), display_students(), find_topper(), search_student(), and class_statistics(); add_student() calls calculate_result()

PYTHON FUNDAMENTALS KEEP THE PROJECT FOCUSED

- **Python:** simple syntax and rapid development for beginners
- **Functions:** divide the program into readable, maintainable modules
- **List:** `students = []` stores the active records

- **Dictionary:** each student record organizes name, m1, and m2 values with related results
- **Menu-driven interface:** makes the application easy to operate
- **No external database:** keeps the project focused on Python fundamentals

SEVEN FUNCTIONS IMPLEMENT THE ANALYZER

- | | | | |
|---|--------------------|---|--------------------|
| 1 | calculate_result() | 2 | add_student() |
| 3 | display_students() | 4 | find_topper() |
| 5 | search_student() | 6 | class_statistics() |
| 7 | main() | | |

Technologies

Technologies: Python 3, lists, dictionaries, functions, loops, conditional statements, and string formatting.

PASS CONDITION

Pass condition: a student passes only when M1, M2, and M3 are each ≥ 40 .